GN2: Transformer-Based Jet Flavour Tagging

A PyTorch Implementation

Based on ATLAS Collaboration — *Nat. Commun.* **17**, 541 (2026)

Goal: train a transformer to classify jets from p_p collisions as b, c, τ, or light from raw tracking information

Why Jet Flavour Tagging?

Physics context

- Jets are collimated sprays of hadrons — the most abundant objects at the LHC
- Identifying the originating quark flavour is critical for:
  Higgs boson measurements ($H \to b\bar{b}$, $H \to c\bar{c}$), top quark physics, BSM searches
- b-hadrons live ~ 1.5 ps $\Rightarrow$ displaced secondary vertex detectable in the inner tracker

Previous approaches

Two-stage pipeline:

1. Low-level algorithms reconstruct displaced vertices from tracks
2. High-level classifier (DL1d, Deep Neural Network) combines outputs

GN2: single end-to-end transformer — no hand-crafted intermediate representations

Four classes

Label	Jet type
b	contains a b-hadron
c	contains a c-hadron, no b
τ	hadronic τ-lepton decay

Key result (ATLAS)

At 70% b-efficiency:
c-jet rejection $\times 3.5$ better than DL1d
light-jet rejection $\times 1.8$ better than DL1d

Dataset & Preprocessing

Input data (HDF5)

- Simulated ATLAS $t\bar{t}$ and Z' events
- **Jet-level:** p_T, η
- **Track-level:** up to 40 tracks/jet
  impact parameters d_0, z_0, angular variables, quality flags
- **Label:** hadron PDG code $\rightarrow \{0, 1, 2, 3\}$

Normalisation (`preprocess.py`)

Computed on **training set only**:

- Jet p_T: log-transform $\rightarrow$ Z-score
- Jet η: Z-score
- Track variables: Z-score per feature

Saved to `norm_stats.json` and reused at eval time

Kinematic selection

$20 < p_T < 250$ GeV $\quad |\eta| < 2.5$

Stratified 70 / 15 / 15% train/val/test split

Efficient I/O (`dataset.py`)

- Indices pre-sorted for **contiguous HDF5 reads**
- `_BatchCollator`: single batched read per step vs. one read per sample
- Invalid tracks filtered via `"valid"` flag; remaining positions zero-padded with

GN2 Architecture

Jet feats $\oplus$ Track feats

$(B, T, n_j + n_t)$

↓

Per-track Initialiser

Linear(256) + ReLU + Linear(256)

↓

Transformer Encoder

4 layers, 8 heads, emb=256, ff=512

↓

Attention Pooling

$\sum_i w_i e_i \rightarrow (B, 128)$

↓

Classification Head

MLP: 128 → 64 → 32 → 4

↓

$(p_b,\ p_c,\ p_u,\ p_\tau)$

Pre-LayerNorm Transformer layer

For each layer ℓ:

$$x' = x + \mathrm{MHA}(\mathrm{LN}(x))$$

$$x'' = x' + \mathrm{FFN}(\mathrm{LN}(x'))$$

Padding mask prevents attention to empty track slots.

Attention pooling

$$w_i = \mathrm{softmax}\left(q^\top e_i\right), \quad g = \sum_i w_i e_i$$

Pads masked to $-\infty$ before softmax.

$\sim$**2.3 M** trainable parameters

Training Pipeline

Loss function

Cross-entropy on 4 classes, labels $\{0, 1, 2, 3\}$
Jets with unmapped labels (-1) are ignored via `ignore_index=-1`

$$\mathcal{L} = -\sum_i y_i \log p_i$$

Training details

- Batch size: 1024 (configurable)
- Gradient clipping: $\|\nabla\|_\infty \leq 1.0$
- Mixed precision (AMP) on CUDA
- Best checkpoint by validation loss
- TensorBoard logging

Optimiser

AdamW, weight decay $\lambda = 10^{-5}$

LR schedule:

1. Linear warmup: $10^{-7} \to 5 \times 10^{-4}$ (first 1% of steps)
2. Cosine annealing: $5 \times 10^{-4} \to 10^{-5}$ (remaining steps)

Code structure (`train.py`)

- `GN2Loss` — loss module
- `lr_scheduler()` — warmup + cosine
- `run_epoch()` — single pass (train or val)
- `train()` — full loop, returns best model
- `TrainingHistory` — stores curves for plotting

Tagging Discriminants

b-tagging: discriminant D_b

$$D_b = \log\left(\frac{p_b}{f_c\, p_c + f_\tau\, p_\tau + (1 - f_c - f_\tau)\, p_u}\right)$$

- $f_c = 0.2$, $f_\tau = 0.05$ (ATLAS defaults)
- $D_b >$ threshold $\Rightarrow$ jet is b-tagged
- Operating point (OP): value of threshold giving a target b-efficiency (e.g. 70%)

c-tagging: discriminant D_c

$$D_c = \log\left(\frac{p_c}{f_b\, p_b + f_\tau\, p_\tau + (1 - f_b - f_\tau)\, p_u}\right)$$

- $f_b = 0.3$, $f_\tau = 0.01$ (ATLAS defaults)
- Same end-to-end model — no retraining needed

ROC curves (background rejection vs. signal efficiency)

For each background class k:

$$\varepsilon_{\text{sig}}(\theta) = \frac{|\{D > \theta\} \cap \text{signal}|}{|\text{signal}|}, \qquad R_k(\theta) = \frac{|\text{background}_k|}{|\{D > \theta\} \cap \text{background}_k|}$$

Evaluation Pipeline (`evaluate.py`)

Outputs produced

1. `metrics.json` — accuracy, per-class P/R/F1
2. `confusion_matrix.pdf` — normalised
3. `score_distributions.pdf` — $p(c|\text{true flavour})$
4. `roc_db.pdf` — b-tagging ROC
5. `roc_dc.pdf` — c-tagging ROC

Usage

```
python -m transformer_jet_tagging.evaluate
  --config configs/config.json
  --device cuda
```

Checkpoint path and output dir read from config.

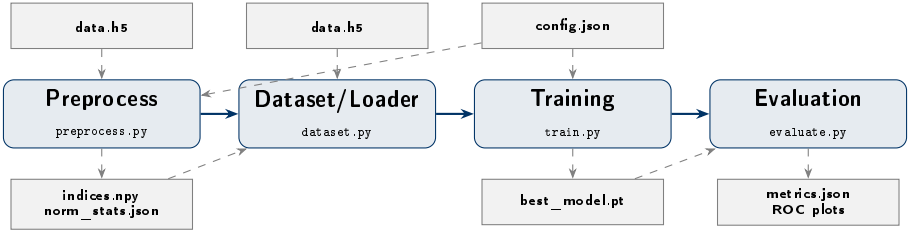
Inference flow

1. Load best checkpoint with `GN2.from_checkpoint()`
2. Iterate test loader, collect softmax probabilities
3. Drop jets with label $= -1$ (unmapped)
4. Compute metrics with `scikit-learn`
5. Compute D_b, D_c and scan thresholds for ROC
6. Save all figures (ATLAS style via `mplhep`)

Results

[To be filled after training]

Full Pipeline at a Glance



preprocess.py

Run once. Saves indices and normalisation stats.

dataset.py

Lazy HDF5 loading, batched reads, masking.

train.py / evaluate.py

Full training loop + test-set evaluation.

Comparison: This Implementation vs. ATLAS GN2

Aspect	ATLAS GN2	This implementation
Framework	PyTorch Lightning	PyTorch (plain)
Architecture	Transformer + 2 aux. heads	Transformer (primary head only)
Aux. objectives	Track origin + vertex grouping	Not implemented
Training data	$\sim$200M jets, 4 folds	Configurable subset
Batch size	12 000	1024 (configurable)
Hardware	NVIDIA A100 ($\sim$300 GPU-h)	Any CUDA/CPU device
Input tracks	40 (sorted by $\lvert d_0/\sigma_{d_0}\rvert$)	40 (sorted by validity)
LR schedule	Warmup + cosine	✓ Identical
Deployment	ONNX Runtime	Checkpoint (.pt)

Main limitation

The two **auxiliary training objectives** (track origin classification and track-pair vertex grouping) are not included. These contribute up to 30% improvement in background rejection according to the ATLAS paper and are the main target for future extensions.

Summary & Next Steps

What has been implemented

- ✓ End-to-end GN2 architecture (transformer + pooling)
- ✓ Optimised HDF5 data pipeline with batched reads
- ✓ Preprocessing with data-leakage-free normalisation
- ✓ Training: AdamW + warmup/cosine LR, AMP, early stopping
- ✓ Evaluation: metrics, confusion matrix, ROC curves
- ✓ Configurable via JSON; all modules standalone-runnable

Known issues to fix

- Bug in `preprocess.py`: double-nested `data_config["data"]`
- `constants.py` / `__init__.py` to be verified
- `label_map` inversion in `evaluate.py`

Future extensions

- **Auxiliary objectives**: track origin + vertex grouping ($\rightarrow$ up to 30% rejection gain)
- `efficiency_at_wp()` for working-point tables
- `pyproject.toml` and `tests/` directory
- 4-fold cross-training (ATLAS production strategy)
- ONNX export for deployment